SOFTWARE REQUIREMENTS SPECIFICATION

for

SIM — Security Incident Mapping Platform

A campus security incident reporting and mapping web platform

Prepared for: SEN202 Weekly Dev — Weekend 1 Assignment
Course Facilitator: Dr. O. O. Ajayi

Prepared by: [Your Name] — Matric No: [Your Matric Number]
Date: 8 July 2026
Version: 1.0

Document Revision History

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for SIM (Security Incident Mapping), a web-based platform that allows students, staff, and landlords within a university campus community to quickly report and view security incidents such as stabbing, rape, theft/stealing, and murder. This document is intended to guide the design, development, and evaluation of the SIM system for the SEN202 Weekend 1 assignment.

1.2 Scope

SIM is a mini web application that enables registered and anonymous users to submit incident reports with a category, description, location, and optional evidence, and to view reported incidents plotted on a map or list view. The system aims to reduce community/campus crime by improving the speed and visibility of incident reporting to relevant stakeholders (campus security, hostel/hall management, and landlords). The initial release focuses on a minimal, low-complexity implementation suitable for a short development window, while the SRS captures the fuller set of requirements envisioned for the platform.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SIM	Security Incident Mapping — the name of the platform being specified.
SRS	Software Requirements Specification.
User	Any student, staff member, or landlord/villa mate using the platform.
Admin	Campus security personnel or system administrator who manages reports.
Incident	A reported security event (e.g. theft, assault, stalking, murder).
GUI	Graphical User Interface.
API	Application Programming Interface.

1.4 References

- SEN202 Weekly Dev — Weekend 1 assignment brief, Dr. O. O. Ajayi (01/07/2026).
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications.

1.5 Overview

Section 2 describes the overall product context. Section 3 lists functional requirements grouped by system feature. Section 4 lists non-functional requirements. Section 5 describes external interface requirements. Section 6 explains how the major components of the system interact.

2. Overall Description

2.1 Product Perspective

SIM is a new, self-contained web-based system. It is not a replacement for formal law-enforcement reporting channels but a complementary rapid-alert and mapping layer that sits between the campus community and campus security/hostel management, improving situational awareness of where and how frequently incidents occur.

2.2 Product Functions

- Allow users to report a security incident with category, description, location, date/time, and optional anonymity.
- Display reported incidents on an interactive map and as a filterable list.
- Allow administrators (campus security) to review, verify, update the status of, and respond to reports.
- Send alerts/notifications to relevant stakeholders when high-severity incidents are reported.
- Provide basic analytics (e.g. incident counts by category, location, and time).

2.3 User Classes and Characteristics

User Class	Characteristics
Student	Primary reporter; occasional user; uses mobile or desktop browser; may need to report anonymously.
Staff	Reports incidents involving staff/campus premises; may also view incident maps for awareness.
Landlord/Villa Mate	Reports incidents occurring in off-campus student housing; less frequent, less technical user.
Administrator (Campus Security)	Reviews, verifies and manages reports; requires a dashboard and moderation tools; technically comfortable.

2.4 Operating Environment

- Client: any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile.
- Server: web server hosting the application backend and database (e.g. Node.js/PHP/Python backend with a relational or document database).
- Mapping: a web mapping service/API (e.g. Google Maps or OpenStreetMap) for geolocation and visualization.

2.5 Constraints

- Short development timeline — the implementation must remain a low-complexity mini web interface.
- Must be deployable and demonstrable from a GitHub repository.
- Sensitive data (identities of reporters, incident details) must be protected.

2.6 Assumptions and Dependencies

- Users have access to an internet-enabled device and a web browser.
- Campus security/administration is willing to monitor and act on reports raised through the platform.
- A third-party mapping API/service is available and accessible.

3. Functional Requirements

The functional requirements are grouped by system feature. Each requirement is uniquely identified for traceability.

3.1 User Registration and Authentication (FR-1.x)

ID	Requirement	Priority
FR-1.1	The system shall allow a user to register with a name, email/phone, and password, or continue as a guest.	High
FR-1.2	The system shall allow a registered user to log in and log out securely.	High
FR-1.3	The system shall allow an administrator to log in via a separate, role-restricted admin portal.	High

3.2 Incident Reporting (FR-2.x)

ID	Requirement	Priority
FR-2.1	The system shall allow a user to submit an incident report specifying category, description, date/time, and location.	High
FR-2.2	The system shall allow the user to choose to report anonymously, hiding their identity from other users and, optionally, from administrators.	High
FR-2.3	The system shall allow the user to attach optional evidence such as a photo.	Medium
FR-2.4	The system shall automatically capture or allow manual entry of the incident's geolocation.	High
FR-2.5	The system shall confirm successful submission of a report to the user.	Medium

3.3 Incident Mapping and Visualization (FR-3.x)

ID	Requirement	Priority
FR-3.1	The system shall display reported incidents as markers on an interactive map.	High
FR-3.2	The system shall allow users to filter displayed incidents by category, date range, and status.	Medium

ID	Requirement	Priority
FR-3.3	The system shall allow users to view incident details in a list view as an alternative to the map view.	Medium

3.4 Administration and Report Management (FR-4.x)

ID	Requirement	Priority
FR-4.1	The system shall allow an administrator to view all submitted reports, including reporter contact details where not anonymous.	High
FR-4.2	The system shall allow an administrator to update the status of a report (e.g. New, Under Review, Resolved).	High
FR-4.3	The system shall allow an administrator to mark a report as verified or false.	Medium

3.5 Notifications (FR-5.x)

ID	Requirement	Priority
FR-5.1	The system shall notify administrators when a new high-severity incident (e.g. stabbing, rape, murder) is reported.	High
FR-5.2	The system should notify the reporting user when the status of their report changes (where the user is not anonymous).	Low

4. Non-Functional Requirements

4.1 Performance

- The system shall load the map/list view within 3 seconds under normal network conditions.
- The system shall support at least 100 concurrent users for the scope of this project demonstration.

4.2 Security

- The system shall encrypt passwords using a secure hashing algorithm and transmit data over HTTPS.
- Anonymous reports shall not store personally identifiable information linking the report to the reporter.
- Only authenticated administrators shall access the admin dashboard and full report details.

4.3 Usability

- The interface shall be simple and intuitive enough for a first-time user to submit a report within two minutes without instructions.
- The system shall be usable on both desktop and mobile screen sizes (responsive design).

4.4 Reliability and Availability

- The system shall be available at least 95% of the time during the evaluation/demonstration period.
- Submitted reports shall not be lost due to a client-side connection failure; the client shall retry or warn the user.

4.5 Scalability and Maintainability

- The system architecture shall separate frontend, backend, and database layers so that components can be scaled or replaced independently.
- Code shall be organized and version-controlled in GitHub to support maintainability and collaboration.

4.6 Legal and Ethical

- The system shall include a disclaimer that it does not replace emergency services and shall provide emergency contact information.
- The system shall comply with basic data-protection practice regarding storage of sensitive personal and incident data.

5. External Interface Requirements

5.1 User Interfaces

- A public-facing web interface with: Home/landing page, Report Incident form, Map/List view, Login/Register pages.
- An Administrator dashboard with: report queue, report detail view, status-update controls, and basic analytics.

5.2 Hardware Interfaces

None beyond a standard internet-enabled client device (desktop, laptop, tablet, or smartphone) with a web browser.

5.3 Software Interfaces

- Mapping/Geolocation API (e.g. Google Maps API or OpenStreetMap/Leaflet) for plotting and geocoding incidents.
- Database system (e.g. MySQL, PostgreSQL, or MongoDB) for persistent storage of users and reports.
- Email/SMS gateway (optional) for administrator alert notifications.

5.4 Communication Interfaces

All client-server communication shall occur over HTTPS using RESTful API calls returning JSON payloads.

6. System Component Interaction

This section explains how the major components of SIM interact to fulfil the requirements above.

6.1 Major Components

Component	Responsibility
Client (Web Frontend)	Renders the GUI (report form, map/list view, login) in the user's browser and communicates with the backend via API calls.
Application Server / Backend API	Handles business logic — authentication, validation of incident reports, status updates — and mediates all access to the database.
Database	Stores user accounts, incident reports, categories, and status history.
Mapping Service (external API)	Provides map rendering, geocoding of addresses, and reverse-geocoding of coordinates for the map/list views.
Notification Service	Sends alerts (email/SMS/in-app) to administrators when high-severity reports are created, and optional status updates to reporters.
Admin Dashboard	A restricted frontend view that consumes the same backend API with elevated permissions to manage reports.

6.2 Interaction Flow — Submitting a Report

1. A user opens the Report Incident form on the Client and fills in category, description, location, and optional evidence/anonymity setting.
2. The Client sends the report data as a REST API request (HTTPS/JSON) to the Application Server.
3. The Application Server validates the input, optionally calls the Mapping Service to geocode the location, and writes the record to the Database.
4. If the report is flagged high-severity, the Application Server calls the Notification Service to alert administrators.
5. The Application Server returns a success response to the Client, which displays a confirmation to the user.

6.3 Interaction Flow — Viewing the Incident Map

6. The Client requests the list of incidents (optionally filtered) from the Application Server.
7. The Application Server queries the Database and returns matching incident records.
8. The Client renders the records as markers using the Mapping Service, or as rows in the list view.

6.4 Interaction Flow — Administrator Review

9. The Administrator logs in through the Admin Dashboard, which authenticates against the Application Server.
10. The Admin Dashboard requests the full report queue from the Application Server.

11. The Administrator updates a report's status; the Admin Dashboard sends the update to the Application Server, which persists it to the Database and, where applicable, triggers the Notification Service to inform the reporter.

7. Appendix

This SRS should be read alongside the accompanying Ishikawa (fishbone) problem-analysis diagram, use-case diagram(s), and the Google Form community questionnaire produced for this assignment, all of which are stored in the project's GitHub repository together with the mini web application and its source code.